DM548: Computer Architecture & System Programming
DM588: Computer Architecture

Fall 2025

Introduction

- Course Overview
- Motivation
- Gates, Memory, Chips, and a Bit of History
- Performance Concepts

People

- Teacher
  Jakob Lykke Andersen
  `jlandersen@imada.sdu.dk`
  `https://imada.sdu.dk/u/jlandersen`
- Teaching Assistant, H9
  Manuel Penschuck
  `penschuck@imada.sdu.dk`
- Teaching Assistant, H1/H8
  Carl-Gustav Øboe Rasmussen
  `crasm@imada.sdu.dk`

What is this course about?

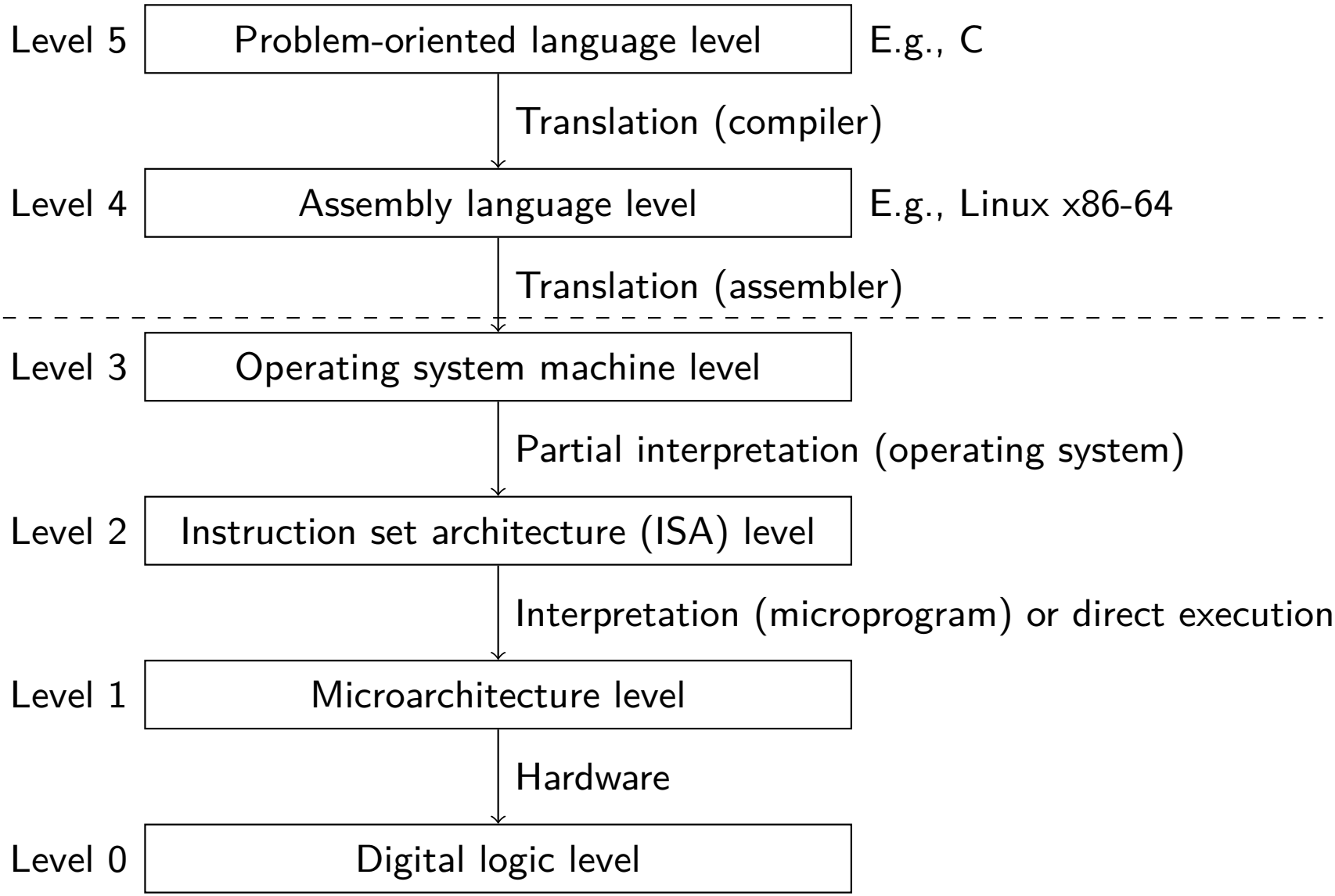
Computer Science: the study of

- the representation of data and
- the manipulation of data.

How are computers realised?

- Architecture: what can it do, as seen by the user.
  E.g., instruction set.
- Organisation: the implementation of an architecture.
  E.g., hardware units, interconnections, memory technology.

Architecture: Levels of Abstraction, Example



Note: the levels are not strictly separated.

Based on [Tanenbaum & Austin, 2013, Fig. 1-2]

Organisation

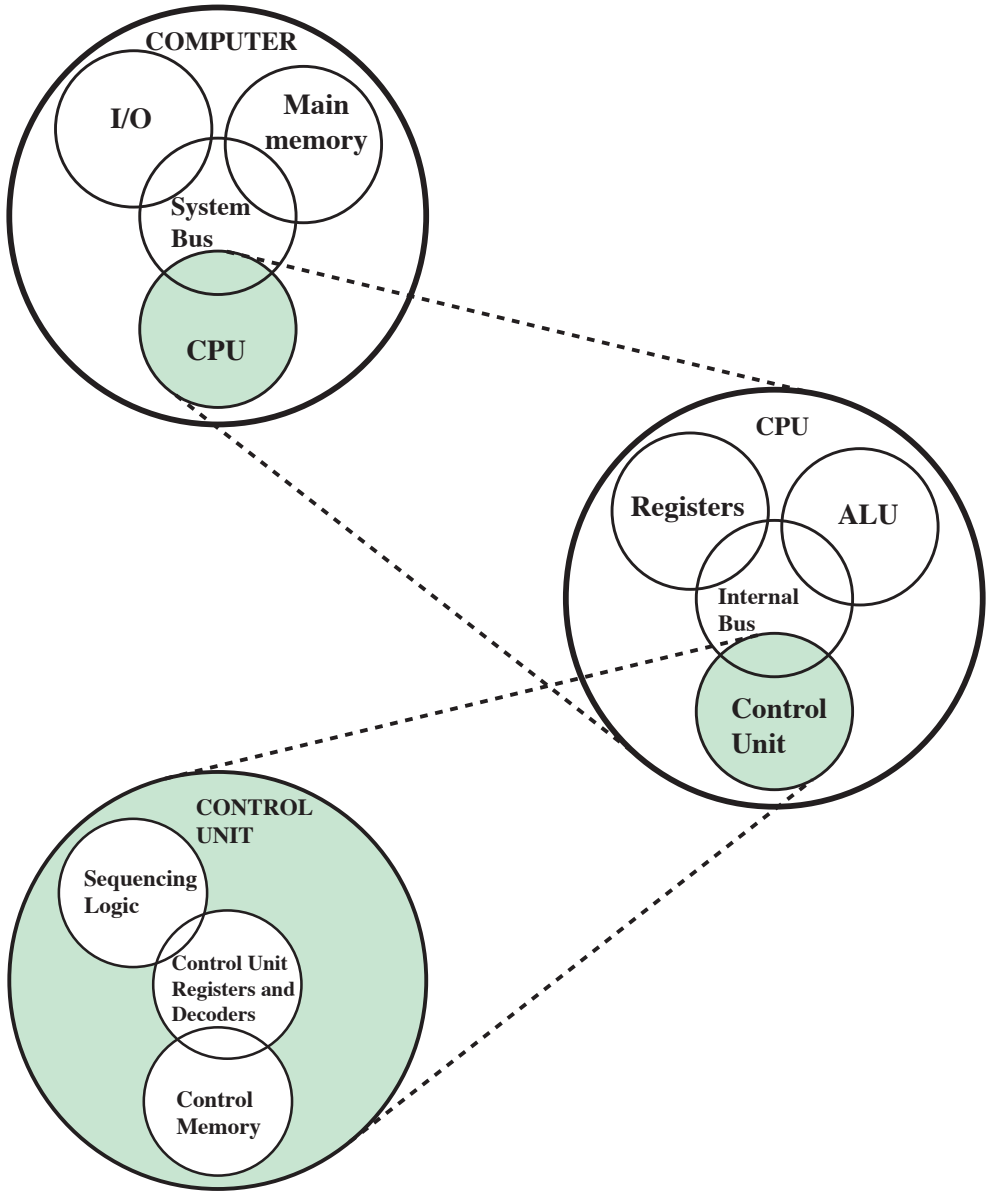


Figure 1.1 A Top-Down View of a Computer

Organisation

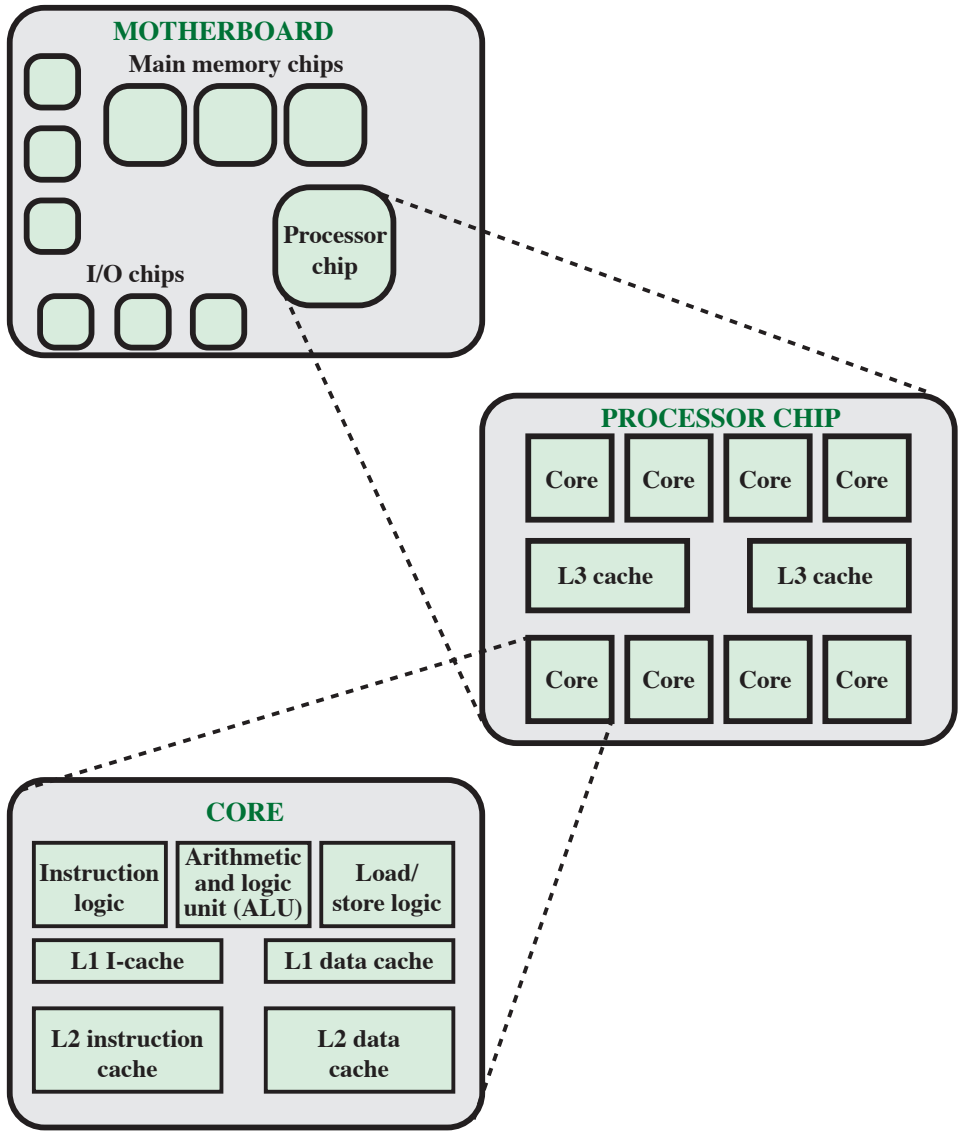


Figure 1.2 Simplified View of Major Elements of a Multicore Computer

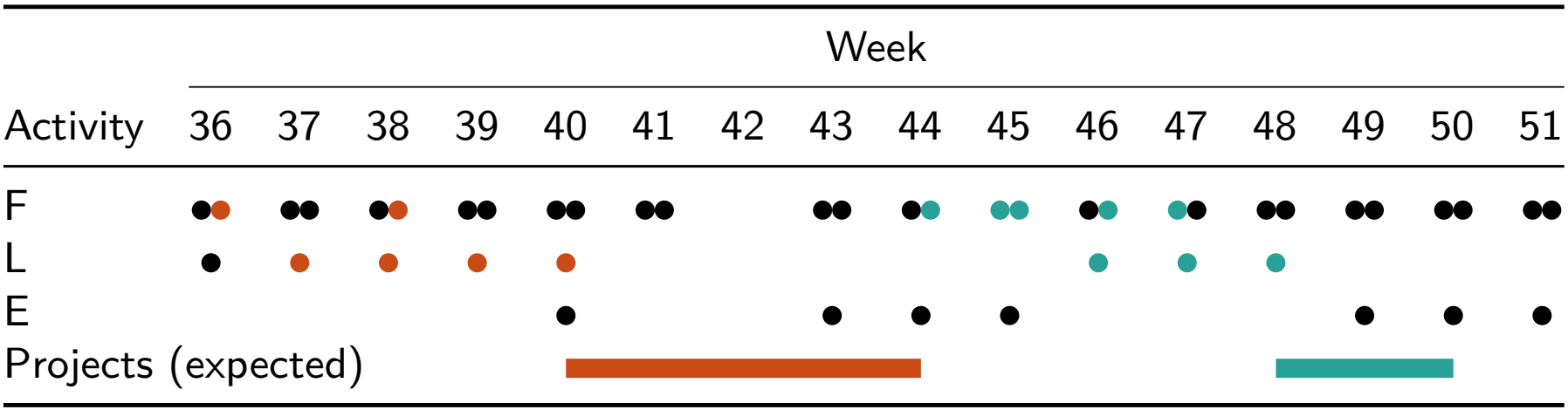
Course Contents (expected order)

- (L2–4) Instruction sets and Linux x86-64 assembly
- Computer arithmetic
- (L0) Digital logic
- Organisation overview and interconnection structures
- Caches
- Internal and external memory
- Input/output
- (L5) System programming in C (not DM588)
- (L1–2) Processor structure and pipelining
- (L1–2) Reduced instruction set (RISC) computers
- (L1–2) Superscalar processors
- (L1–2) Parallel processing and multicore computers
- (L0–2) Control unit and microprogramming

Course Organisation

- Workload:
  - DM548: 10 ECTS points
  - DM588: 7.5 ECTS points
- Sessions
  - Lectures (DM548: 21, DM588: 16)
  - Lab sessions (DM548: 8, DM588: 5): training for the projects
  - Exercise sessions (7): training for the written test
- Projects
  - Project with assembly programming
    (Expected deadline: late October)
  - (not DM588) Project with C programming
    (Expected deadline: mid December)
- Evaluation
  - 7-scale grade
  - Based on a written test in early January,
  - and the project(s).
  - See also the course descriptions (link in itslearning).

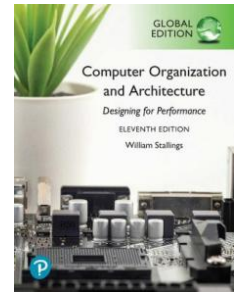
Course Timeline (Expected)



- : assembly
- : C (not DM588)
- The schedule has several lectures and 1 exercise session too many. The sessions to be cancelled will be selected later in the semester.

Resources

- Course website: itslearning
- Text book:
  William Stallings:
  "Computer Organization and Architecture",
  Global Edition, 11th edition, Pearson, 2012.
  (Can be bought in the SDU bookstore)



- Attending lectures, labs, and exercise sessions
- Solving exercises
- Questions:
  1. Check itslearning
  2. Ask your TA
     (ask them about preferred communication method)
  3. Ask me
     - Via email (most reliable method)
       If you are asking about programming: send text as text, not screenshots, to enable copy-pasting.
     - Come by my office
       (see `https://imada.sdu.dk/u/jlandersen`)
     - Via itslearning messages(?) (expect higher response time)

"10 (7.5) ECTS points"?

A measure of workload (not difficulty):

- 60 ECTS points $\equiv$ 1 year full-time work $\equiv$ 1650 hours
- $\Rightarrow$ 1 ECTS point $\equiv$ 27.5 hours

This course:

- 10 (7.5) ECTS points $\equiv$ 275 (206.25) hours
- Weeks: 15 teaching, 2 without (42, 52), 1-2 pre/post
  $\Rightarrow$ 14.5–15.3 (10.9–11.5) hours per week, on average
- What is your study strategy?
- Example strategy:

Activity	Hours (DM548)	Hours (DM588)
Lectures + pre/post work	$21 \cdot (2+3) = 105$	$16 \cdot (2+3) = 80$
Exercises + preparation	$7 \cdot (2+3) = 35$	$7 \cdot (2+3) = 35$
Labs + pre/post-work	$8 \cdot (2+2) = 32$	$5 \cdot (2+2) = 20$
Projects	$40 + 25 = 65$	35
Exam + preparation	$4 + 28 = 32$	$4 + 28 = 32$
Remaining	6	4.25

- Course Overview
- Motivation
- Gates, Memory, Chips, and a Bit of History
- Performance Concepts

Why this course?

- It's Computer Science, we are Computer Scientists, so ...
- Some later courses depends on this one.
- Other courses usually deals with computation on an abstract level. Here we explore how to build the basis of those abstractions.
- Our computers don't implement the usual mathematical operators.
- Performance is not only determined by algorithmic complexity.
- Many bugs arise due to architectural properties.
- Many (most?) security problems arise on this level as well.

Arithmetic Operations

- Basic mathematical numbers:
  - integers, $\mathbb{Z}$
  - reals, $\mathbb{R}$
- Basic data types: `int` and `float`
- Is $x \cdot x \geq 0$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `float`: yes

Arithmetic Operations

- Basic mathematical numbers:
  - integers, $\mathbb{Z}$
  - reals, $\mathbb{R}$
- Basic data types: `int` and `float`
- Is $x \cdot x \geq 0$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `float`: yes
  - `int`: sometimes(!)
    - $40\,000 \cdot 40\,000 = 1\,600\,000\,000$
    - $50\,000 \cdot 50\,000 = ?$

Arithmetic Operations

- Basic mathematical numbers:
  - integers, $\mathbb{Z}$
  - reals, $\mathbb{R}$
- Basic data types: `int` and `float`
- Is $x \cdot x \geq 0$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `float`: yes
  - `int`: sometimes(!)
    - $40\,000 \cdot 40\,000 = 1\,600\,000\,000$
    - $50\,000 \cdot 50\,000 = ?$
- How about associativity, is $(x + y) + z = x + (y + z)$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `int`: yes (assuming no overflow)

Arithmetic Operations

- Basic mathematical numbers:
  - integers, $\mathbb{Z}$
  - reals, $\mathbb{R}$
- Basic data types: `int` and `float`
- Is $x \cdot x \geq 0$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `float`: yes
  - `int`: sometimes(!)
    - $40\,000 \cdot 40\,000 = 1\,600\,000\,000$
    - $50\,000 \cdot 50\,000 = ?$
- How about associativity, is $(x + y) + z = x + (y + z)$?
  - $\mathbb{Z}$, $\mathbb{R}$: yes
  - `int`: yes (assuming no overflow)
  - `float`: sometimes(!)
    - $(10^{20} - 10^{20}) + 3.14 = 3.14$ (well, actually . . .)
    - $10^{20} + (-10^{20} + 3.14) = ?$

We assume `int` and `float` as defined in C.

Arithmetic Operations

The bad news:

- Our computers do not have infinite representations, so
- `int` $\not\equiv \mathbb{Z}$
- `float` $\not\equiv \mathbb{R}$

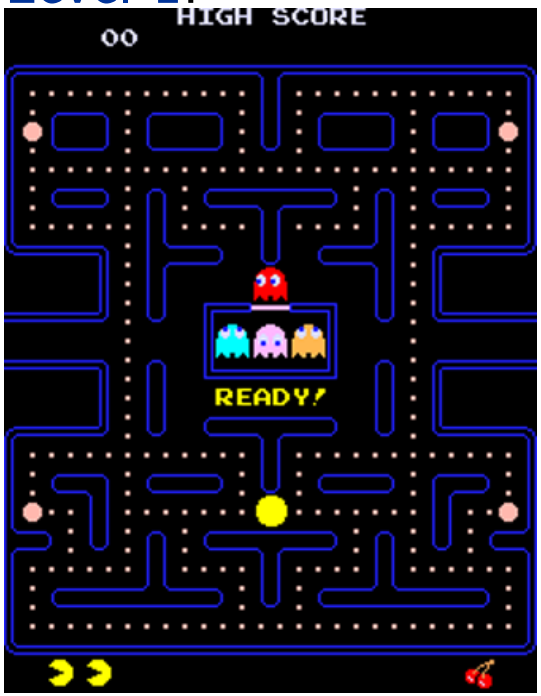
The good news:

- The results are not arbitrary, we can still describe them mathematically.
- `int` in C is an approximation of $\mathbb{Z}$
- `int` in Java is a ring (wrap around on overflow/underflow)
- `float` is an approximation of $\mathbb{R}$

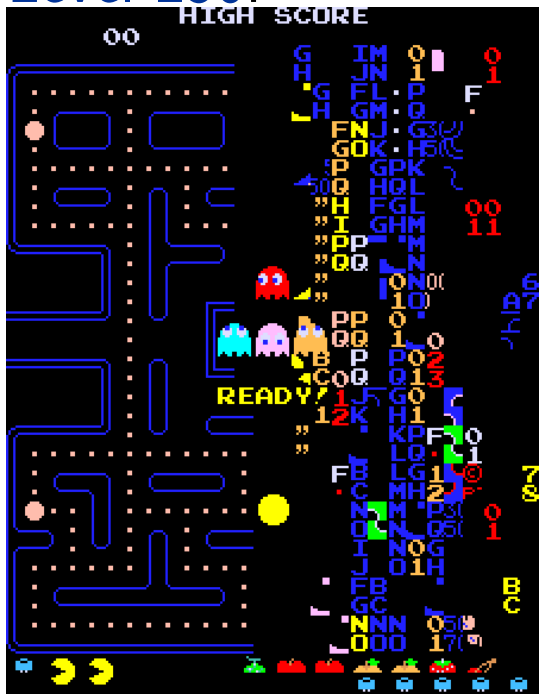
If we do not know our data types we are bound to make funny/horrible/expensive/dangerous errors.

Pacman: the Split Screen

Level 1:

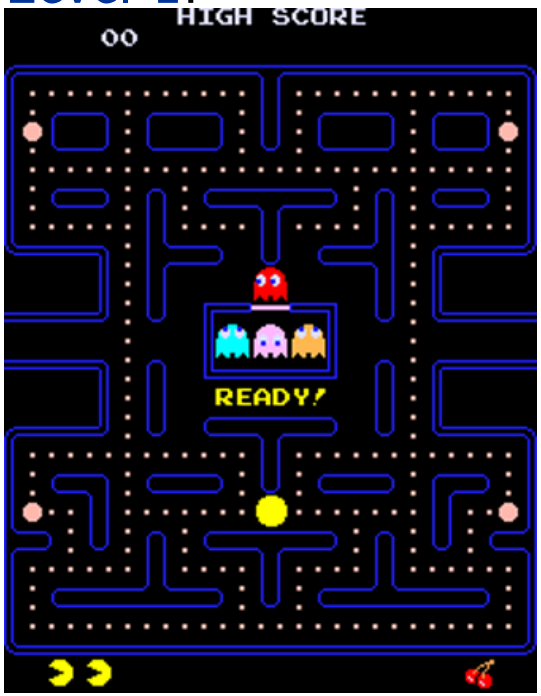


Level 256:

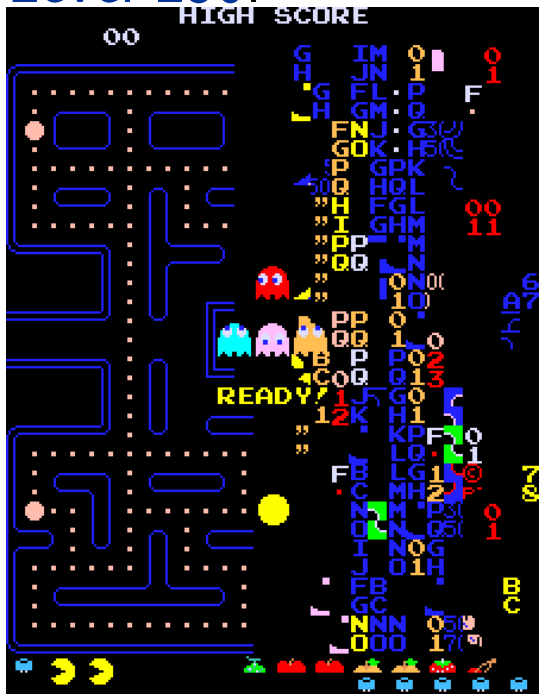


Pacman: the Split Screen

Level 1:



Level 256:



- Levels are 0-indexed and stored in 8-bit unsigned representation. That is, level 1 is number 0, and level 256 is number 255.
- The logic for drawing fruits assumes $\langle levelNumber \rangle + 1 > \langle levelNumber \rangle$.
- But $255 + 1 = 0$, so it tries to draw 256 fruits.

[https://pacman.holenet.info]

[http://www.donhodges.com/how_high_can_you_get2.htm]

Patriot Missle Air Defense System

- Supposed to detect incoming missiles and shoot them down.
- In Dhahran, Saudi Arabia, 25 February, 1991 (during the Golf War), it failed in detecting a Scud missile.
- The Scud hit army barracks, killing 28 US soldiers and injuring another 98.

Why did it fail?

Patriot Missle Air Defense System

- Supposed to detect incoming missiles and shoot them down.
- In Dhahran, Saudi Arabia, 25 February, 1991 (during the Golf War), it failed in detecting a Scud missile.
- The Scud hit army barracks, killing 28 US soldiers and injuring another 98.

Why did it fail? A software bug.

- It uses a radar system to detect missiles.
- So it must keep track of time to predict where they are going.
- The internal time was counting number of tenths of seconds.
- What is the time in seconds? $0.1 \cdot \langle time \rangle$
- This was performed in 24-bit fixed point arithmetic,
- but 0.1 can not be represented exactly in that format.

[https://en.wikipedia.org/wiki/MIM-104_Patriot#Failure_at_Dhahran]

Patriot Missle Air Defense System

- The system was not supposed to be running a long time,
- but Scuds are fast ($1676\,\frac{\mathrm{m}}{\mathrm{s}}$) and fly a short time,
- so the system had been kept on for around 100 hours.
- The time error was now around 0.34 seconds,
- corresponding to more than half a kilometre in travel distance.
- The system could not find the Scud at the predicted area,
- so it decided that the first detection must have been an error.

[http://www-users.math.umn.edu/~arnold//disasters/patriot.html]

Ariane 5 Launch Failure

On 4 June, 1996 an Ariane 5 rocket disintegrated 40 s after launch.

- Its Inertial Reference System (SRI) was essentially reuse from Ariane 4,
- but the flight paths were very different.
- The horizontal velocity, represented in 64-bit floating point, was much higher in Ariane 5.
- During calculations it was converted to 16-bit signed integer format,

Ariane 5 Launch Failure

On 4 June, 1996 an Ariane 5 rocket disintegrated 40 s after launch.

- Its Inertial Reference System (SRI) was essentially reuse from Ariane 4,
- but the flight paths were very different.
- The horizontal velocity, represented in 64-bit floating point, was much higher in Ariane 5.
- During calculations it was converted to 16-bit signed integer format,
- but it overflowed, triggering an operand error, causing diagnostics data to be output.
- Both SRIs had this problem, so the flight computer started reading the diagnostics data as if it was flight data.
- It thought it was out of course and adjusted the engines to correct it.
- The angle of attack became too large, so the boosters broke off, which in turn triggered the self-destruct system.

[`http://sunnyday.mit.edu/nasa-class/Ariane5-report.html`]

Memory in Practice

- Memory is not unbounded, it must allocated and managed.
  - High-level languages may help or completely take care with this,
  - but it may not know what is best to do in all cases.
- Program performance is often limited due to memory.
- Memory referencing bugs are especially bad.
  - Their effect may occur much later.
  - Their effect may occur in a completely unrelated code.
  - They can have severe security implications.
- Memory is not just memory.
  - Cache and virtual memory effects are complicated.
  - Adapting programs to the characteristics of the memory system can lead to major performance improvements.

Example: Copying a Matrix

How do we copy a matrix?

How about row by row:

```
1 foreach row i do
2     foreach column j do
3         Copy cell (i, j).
```

but we could instead do column by column:

```
1 foreach column j do
2     foreach row i do
3         Copy cell (i, j).
```

On most machines one way is much faster than the other.

Practical Performance

Asymptotic complexity is important for understanding the nature of computation and the bigger picture.

What does $O(n \log n)$ mean?

- It represents the asymptotic count of some operation.
  - Are we counting the right operation?
  - What about other operations?
  - Which matter in practice?
- It may have any constant factor and less significant terms:
  - $10^{100} \cdot n \log n$
  - $n \log n + 10^{100} \cdot n$

Which algorithm to use?

Practical Performance

Asymptotic complexity is important for understanding the nature of computation and the bigger picture.

What does $O(n \log n)$ mean?

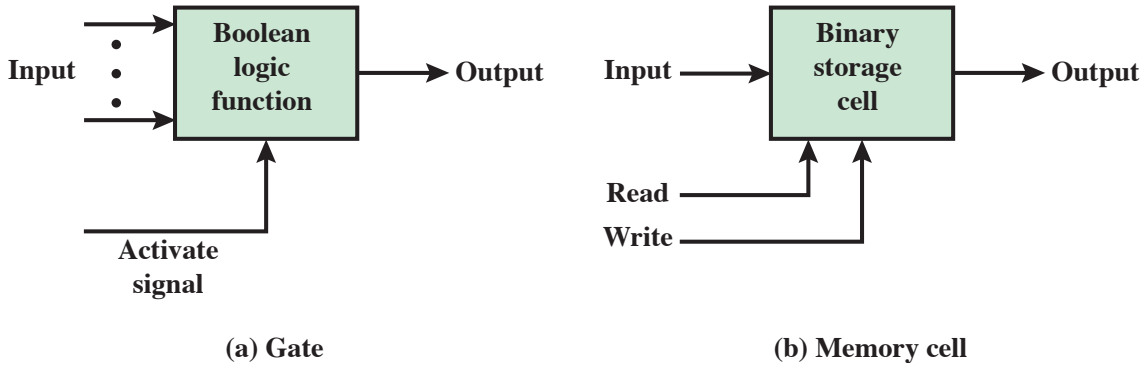
- It represents the asymptotic count of some operation.
  - Are we counting the right operation?
  - What about other operations?
  - Which matter in practice?
- It may have any constant factor and less significant terms:
  - $10^{100} \cdot n \log n$
  - $n \log n + 10^{100} \cdot n$

Which algorithm to use?

- What is the expected use case?
- What is the complexity of the algorithm?
- How does the algorithm transform into real code?
- How is the code compiled to an executable program?
- How do we evaluate performance (in our use case)?
- How can we improve performance?

- Course Overview
- Motivation
- Gates, Memory, Chips, and a Bit of History
- Performance Concepts

Gates and Memory Cells



- Data: two signal values, 0 (FALSE) and 1 (TRUE).
- Data processing (gates): boolean logic (AND, OR, NOT, ...)
- Data storage (memory cells): readable, writeable
- Data movement: connections between combinations of gates and memory cells.

Remember the two topics?

- Architecture: how do can we program the processing to do what we want?
- Organisation: how can we build larger and more advanced processing and storage components?

ENIAC

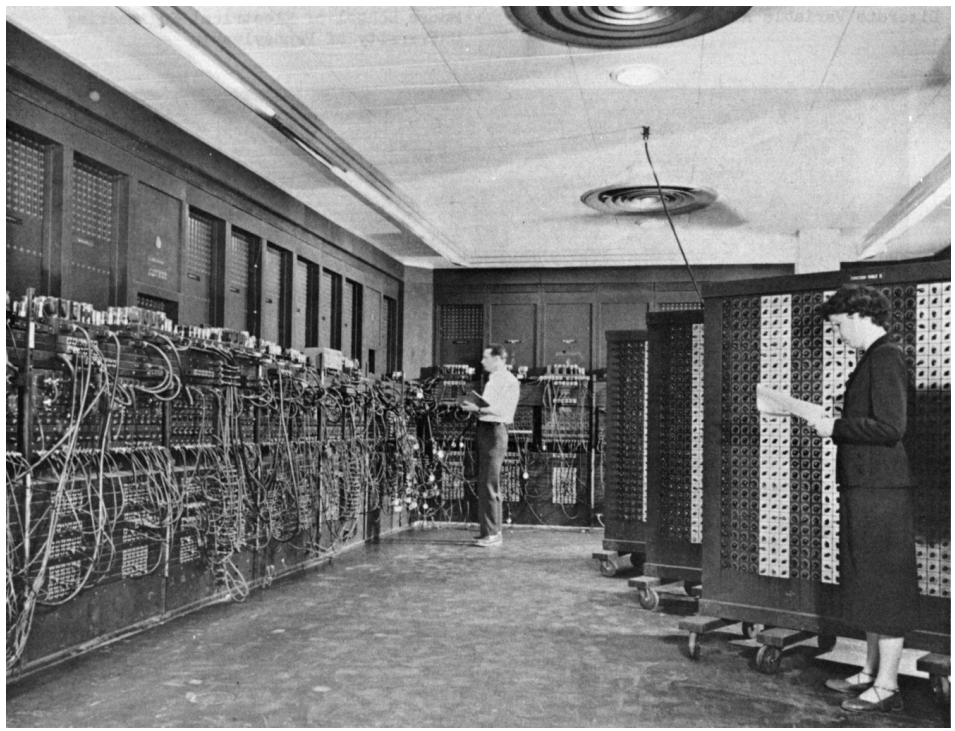
- "Electronic Numerical Integrator and Computer"
- The first electronic general-purpose computer.
- Designed to calculate artillery firing tables.
- Completed in 1945 at the University of Pennsylvania.
- Contained: 20 000 vacuum tubes, 70 000 resistors, and 10 000 capacitors.
- Dimensions: $2.4\,\text{m} \times 0.9\,\text{m} \times 30\,\text{m}$, occupying $167\,\text{m}^2$
- Energy consumption: 150 kW
- Consisted of panels, each with a specific purpose, e.g.,
  - Initiating unit, starting and stopping the machine.
  - Master programmer, the overall orchestration.
  - Cycling unit, generating the internal clock signal (100 kHz).
  - 20 accumulator units, holding 10-digit numbers (i.e., not binary representation).
    Could perform 5000 simple additions/subtractions per second.
  - Reader unit and print unit (punch cards).

[https://en.wikipedia.org/wiki/ENIAC]

ENIAC

- Programming: set switches and connect wires.
- Mapping a program to the machine could take weeks.
- Actually setting up a program could take days.
- Vacuum tubes would frequently burn out when running.
- Had no persistent memory, data must be printed.

Important note: program and data are separate entities, a program is the physical setup of the machine.

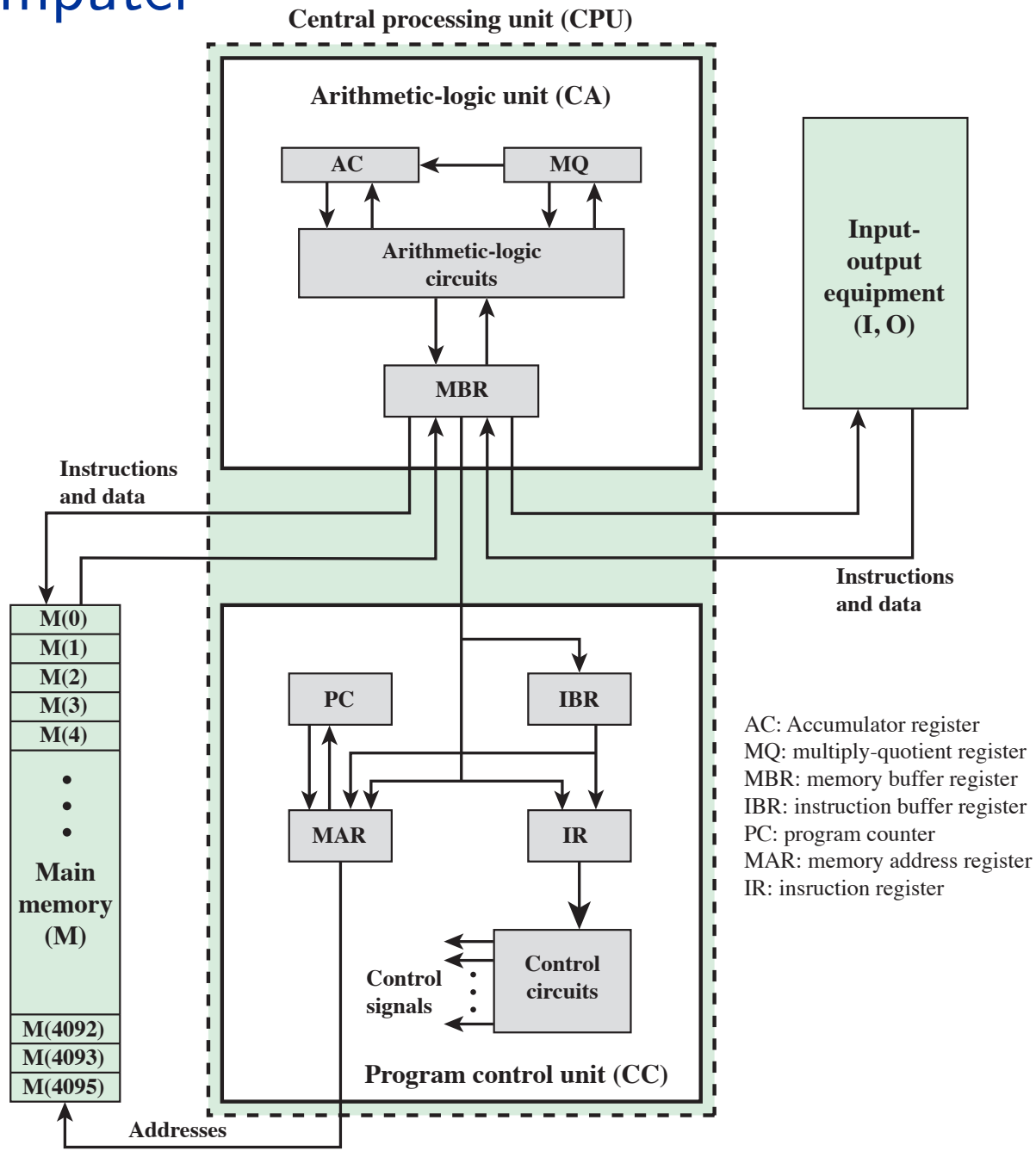


[https://en.wikipedia.org/wiki/ENIAC]

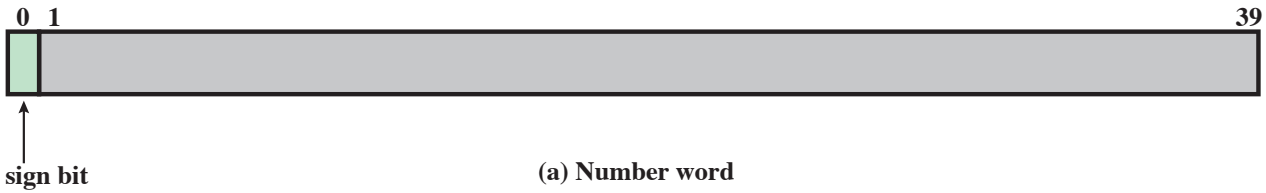
John von Neumann and the IAS Computer

- IAS: Institute for Advanced Study, Princeton, New Jersey.
- In 1945 von Neumann proposed a fundamental new design: Encode the program as data, the stored program concept.
- Now known as the von Neumann architecture.
- Basically all computers are now based on this idea.
- Components:
  - Memory, storing both data and program instructions.
  - Central processing unit (CPU):
    - Arithmetic and logic unit (ALU), operating in binary.
    - Control unit, interpret instructions from memory and causes their execution.
  - Input/output (IO) equipment, operated by the control unit.

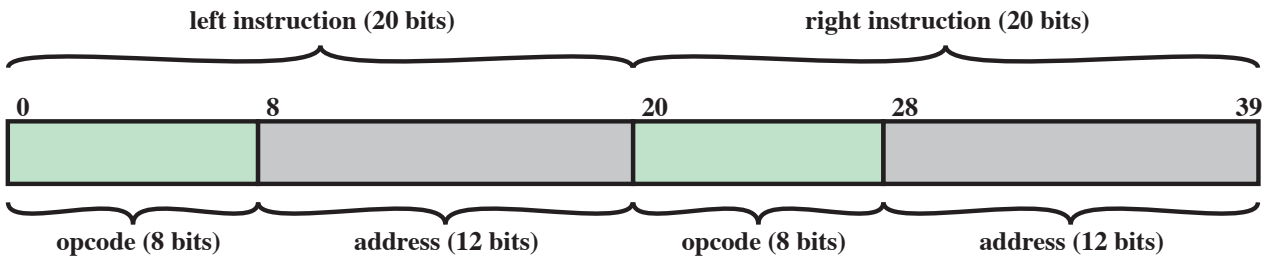
The IAS Computer



IAS Memory Formats



(a) Number word



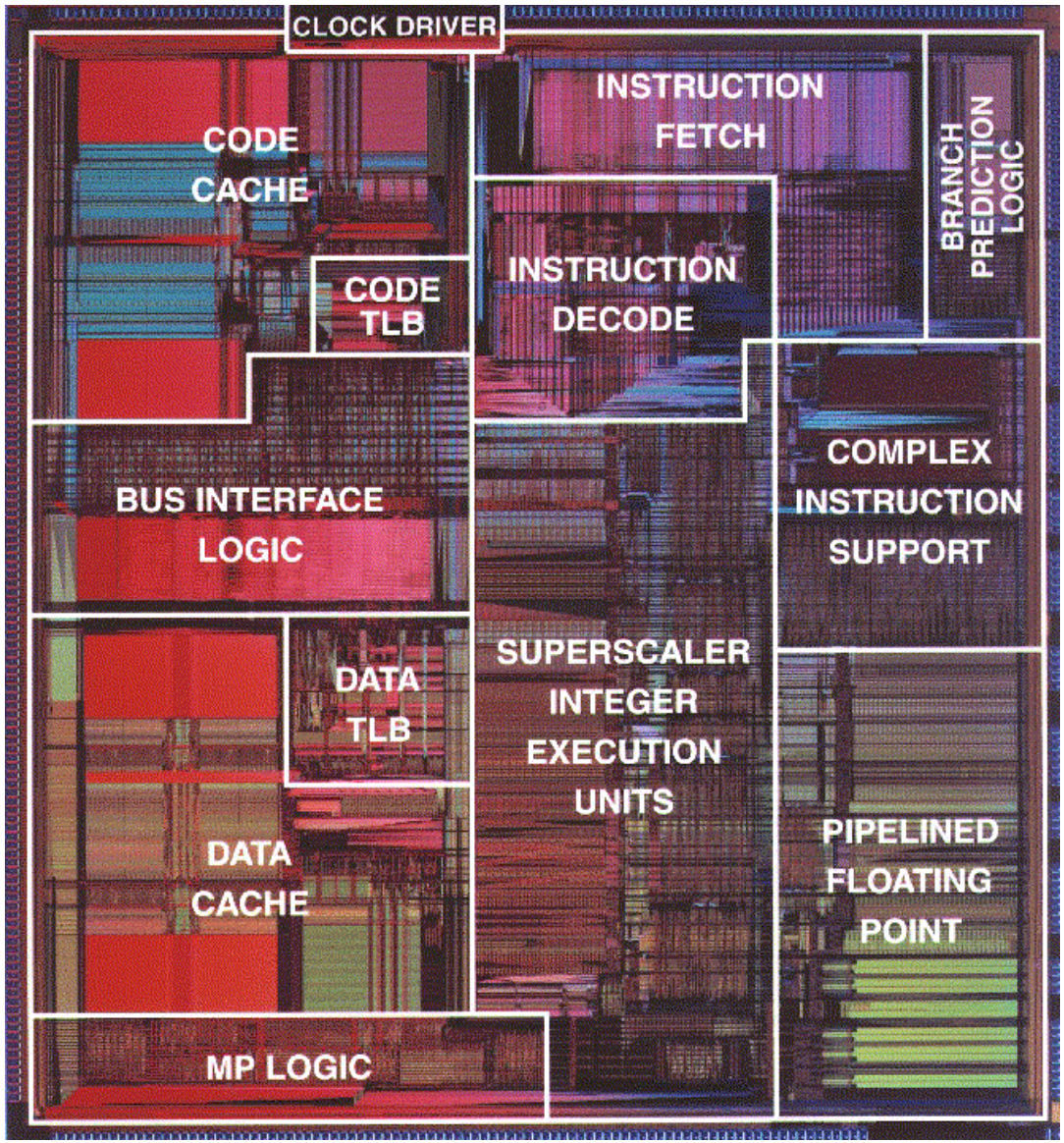
(b) Instruction word

- The same 40 bits of a word can be interpreted in two ways.
- The data does not describe what it is.
- The conventions of the machine decides how to interpret it.

Important note: we will do this a lot;
Interpreting the same bits in different ways depending on context.

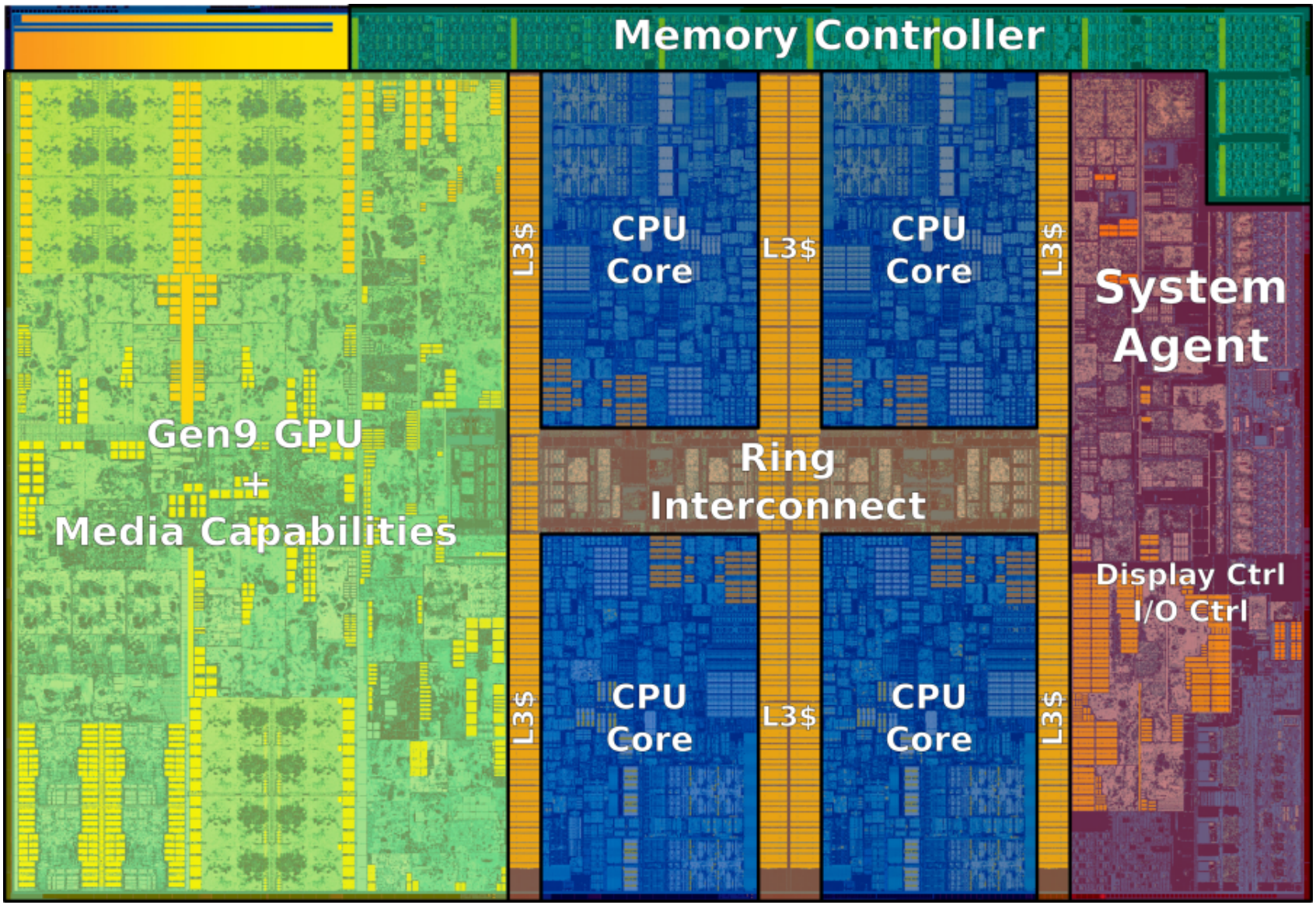
We do this in everyday life as well, e.g., course name “DM548”, plane seat “12A”, SDU room “Ø13-605b-2”.

Intel Pentium Pro (1995)



[https://people.cs.clemson.edu/~mark/330/colwell/pentium.gif]

Intel Skylake (2015)



[https://en.wikichip.org/wiki/intel/microarchitectures/skylake_(client)]

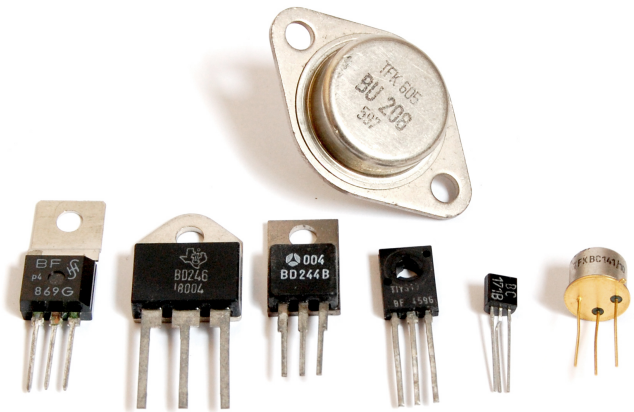
Computer Generations

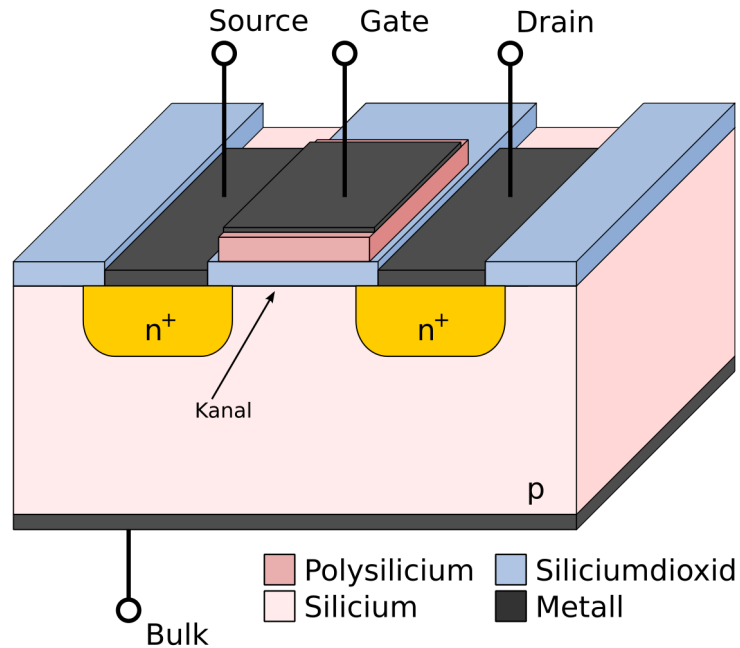
Generation	Approximate Dates	Technology	Typical Speed (Ops. per second)
1	1946–1957	Vacuum tubes	40 000
2	1958–1964	Transistors	200 000
3	1965–1971	Small- and medium-scale integration	1 000 000
4	1972–1977	Large-scale integration	10 000 000
5	1978–1991	Very large-scale integration	100 000 000
6	1991–	Ultra large-scale integration	1 000 000 000

- "Integration" means that transistors are integrated in a silicon chip, and not on a circuit board.
- A new generation starts when the fundamental hardware technology changes.
- But after generation 3 the classification becomes less meaningful due to rapid development.
- Creating multichip modules has recently become more popular.

1947

- The transistor is invented at Bell Labs.
- Smaller, cheaper, less energy, and more reliable than vacuum tubes.





[https://de.wikipedia.org/wiki/Transistor]

1952–1964

IBM mainframes:

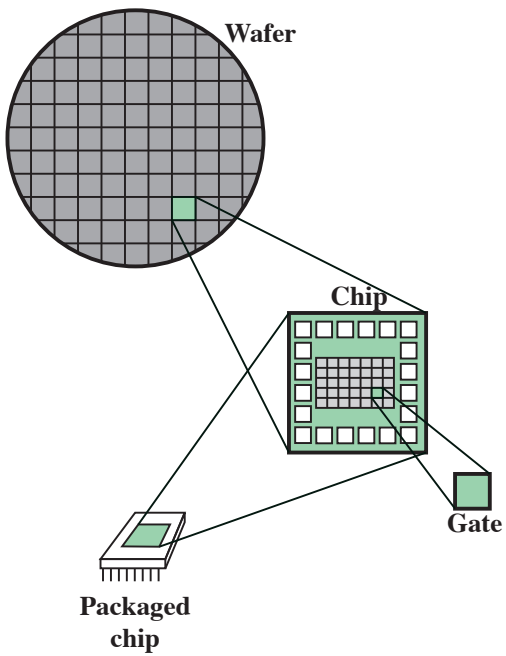
- 700 series, vacuum tubes, based on the IAS.
- 7000 series, discrete transistors.

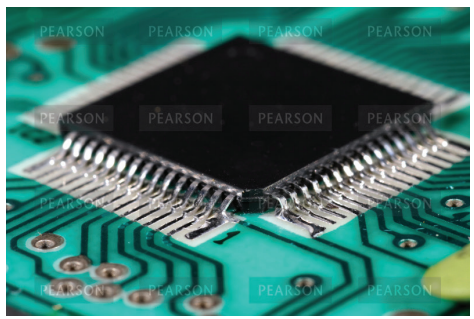
IBM became a marked leader for business computers.

[https://en.wikipedia.org/wiki/IBM_700/7000_series]

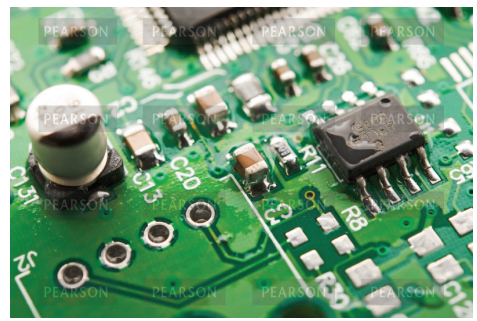
The Third Generation (1965–1971)

- Instead of connecting individual transistors to circuit boards, the integrated circuit was invented.
- Package multiple transistors in one chip, and connect that to the circuit boards.
- Important computers in this era:
  IBM System/360, DEC PEP-8





(a) Close-up of packaged chip



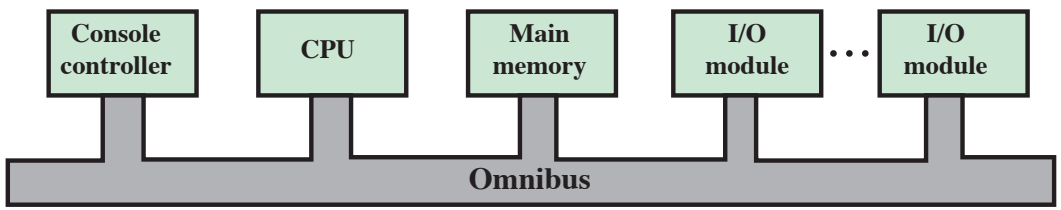
(b) Chip on motherboard

IBM System/360 (1964)

- First planned family of computers.
- All variants basically use the same instruction set and operating system.
- The hardware can be upgraded without changing the software!
- IBM became overwhelmingly dominant (70 % marked share).
- IBM mainframes today still follow the 360 architecture.

DEC PEP-8 (1964)

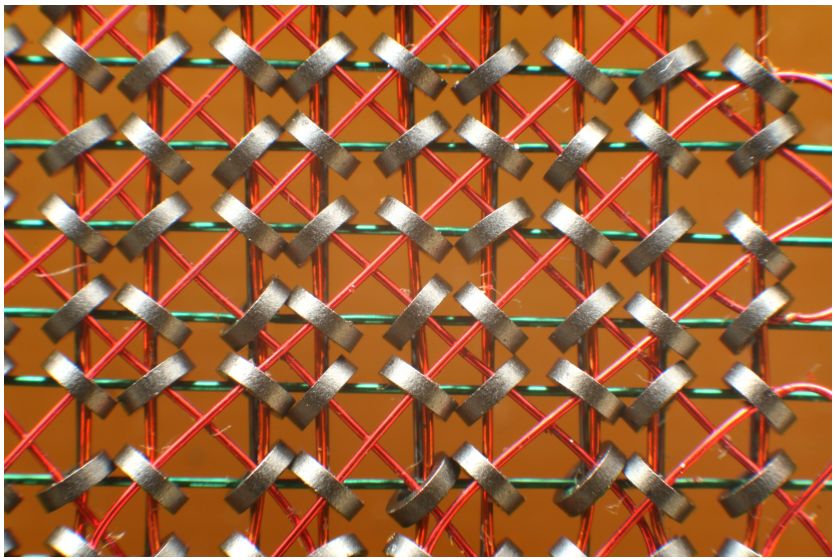
- The first (relatively) small computer.
- It could fit one a lab bench and did not require air condition!
- Relatively cheap: \$16 000
- Other manufacturers bought it and integrated it into a total system for resale. OEM: Original Equipment Manufacturer.
- Deviation from the von Neumann architecture: the bus structure. High flexibility.

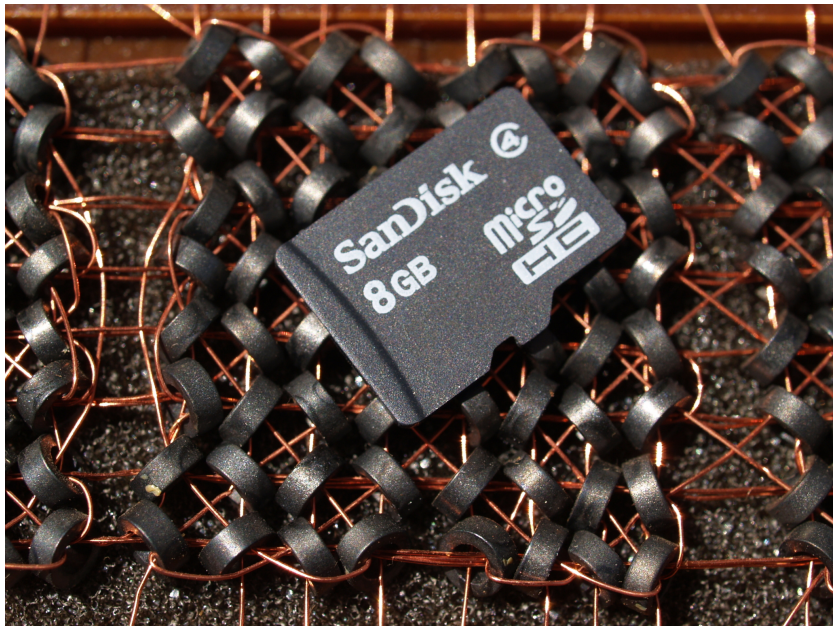


- Bus structures have been widely used until the '00s.

Semiconductor Memory

- In the ‘50s and ‘60s most memory was core memory.





8 B vs. 8 GB

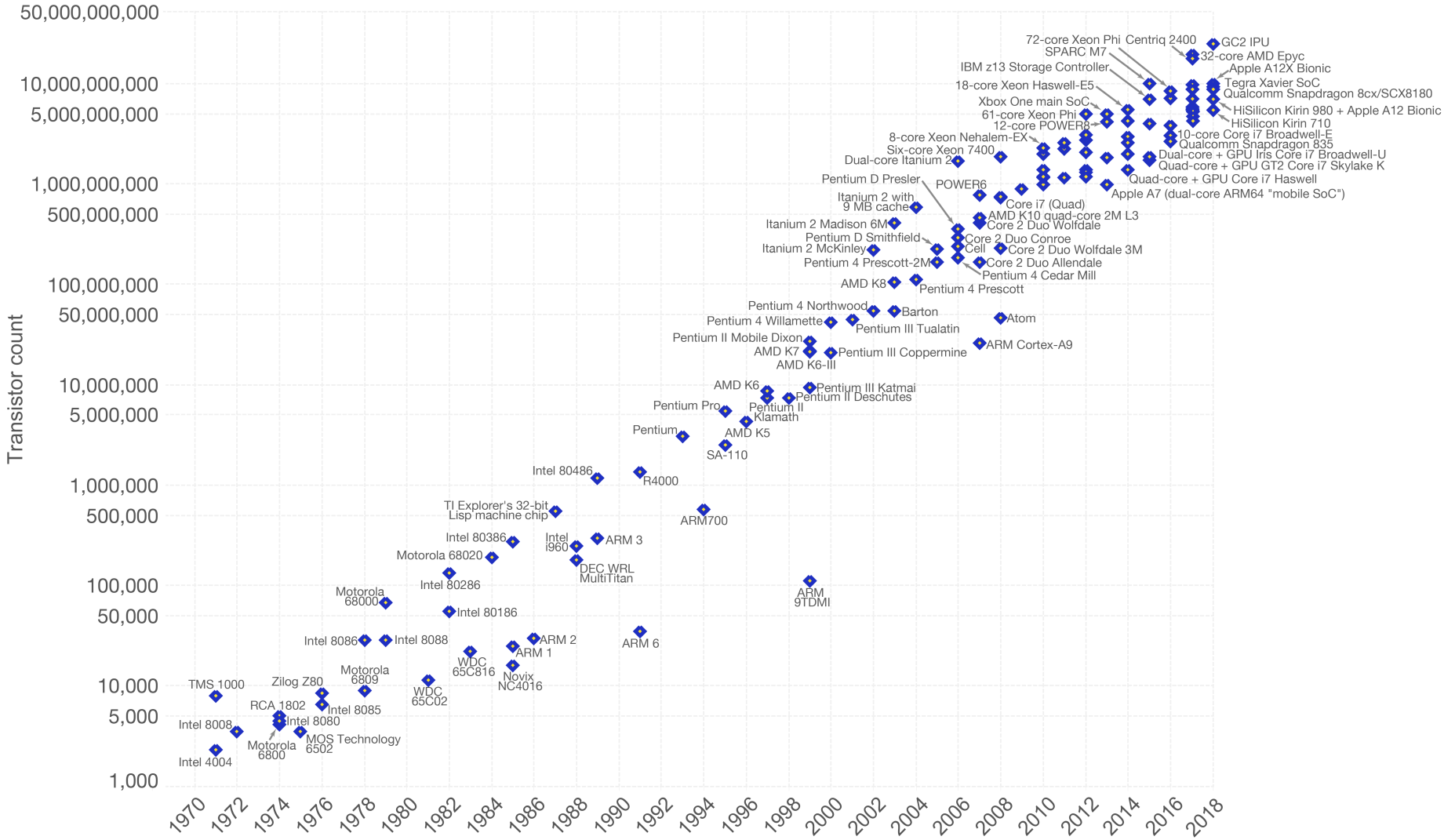
- In 1970 the first semiconductor chip with 256 bits memory was constructed.
- In 1974 semiconductor memory became cheaper per bit.

[https://en.wikipedia.org/wiki/Magnetic-core_memory]

Moore's Law

- In 1965 the Intel co-founder Gordon Moore predicted that the number of transistors per chip would double every year.
- To the surprise of many, he was almost right: it has doubled every 18 months.
- Impact:
  - The price per chip has remained unchanged.
  - Closer packaging of transistors leads to shorter paths, making higher operating speeds possible.
  - Computers become smaller, making them more convenient en more places.
  - Reduced power consumption, and therefore also cooling requirement.
  - Internal connections in chips are much more reliable than external soldered connections. More circuitry per chip means fewer soldered interconnections.

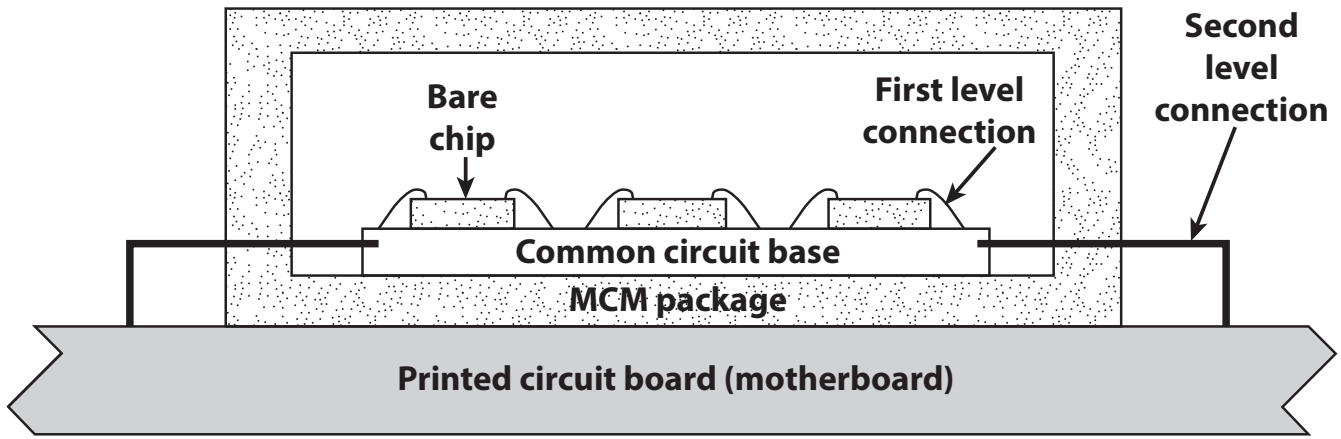
Moore's Law



[https://en.wikipedia.org/wiki/Moore's_law]

Multichip Modules

- Making larger and larger chips is not a good solution.
- A single flaw ruins the whole chip.
- Creating multiple smaller ordinary chips makes for slow interconnection.
- Instead: create smaller chips ("chiplets") and package them together.

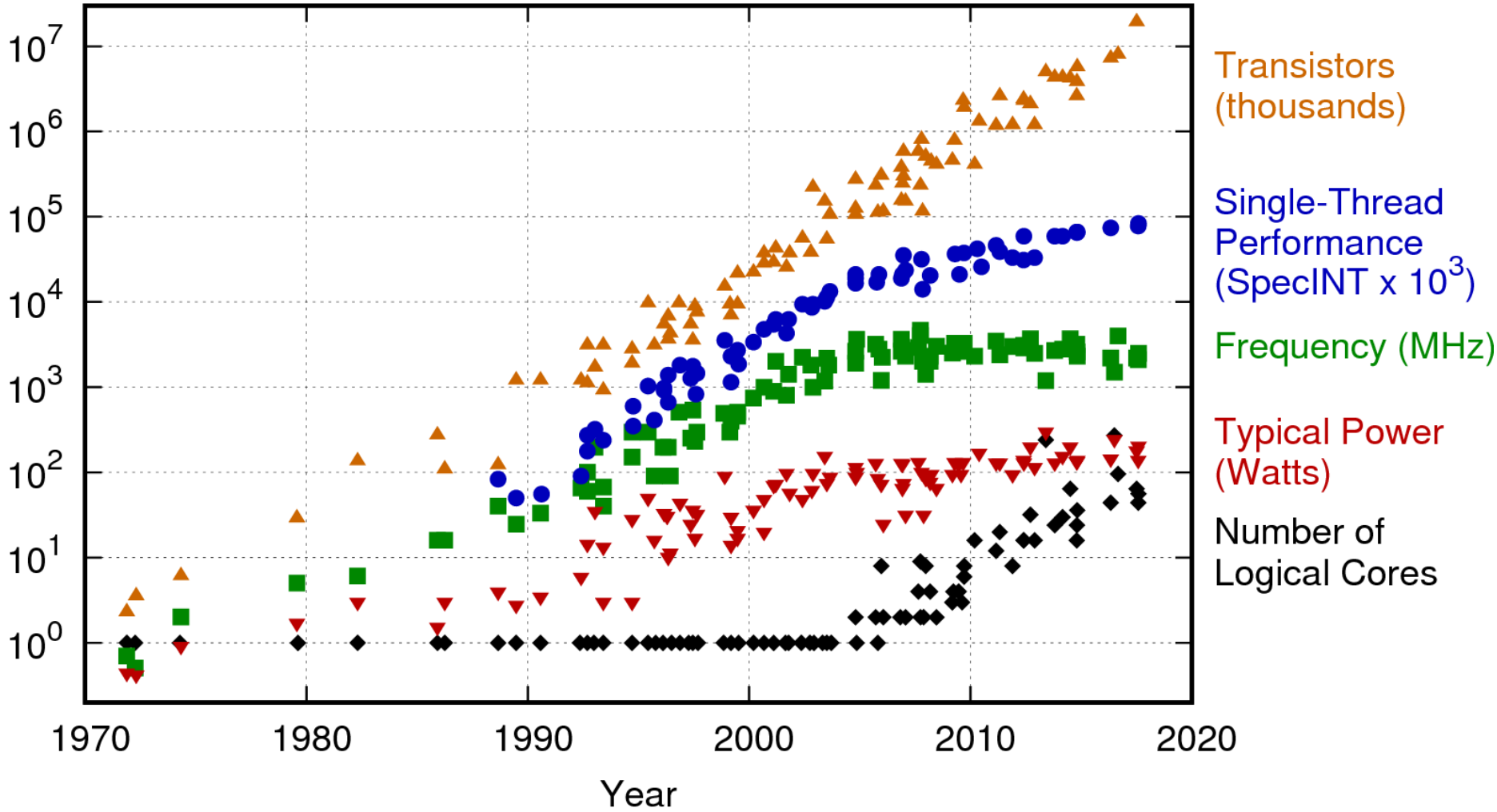


Intel x86 Architecture (selected chips)

Chip	Year	Speed (MHz)	Memory	Note
4004	1971	0.180	640 B	First microprocessor on a chip.
8008	1972	0.180	16 kB	First 8-bit processor.
8080	1974	2	64 kB	First general-purpose CPU on a chip.
8086	1978	5–10	1 MB	First 16-bit CPU on a chip.
8088	1979	5–10	1 MB	Used in IBM PC.
80286	1982	8–12	16 MB	Memory protection present.
80386	1985	16–33	4 GB	First 32-bit CPU.
80486	1989	25–100	4 GB	Built-in 8-kB cache mem.
Pentium	1993	60–233	4 GB	Two pipelines; later had MMX.
Pentium Pro	1995	150–200	4 GB	Two levels of cache built in.
Pentium II	1997	233–450	4 GB	Pentium Pro plus MMX instructions.
Pentium III	1999	650–1400	4 GB	SSE Instructions for 3D graphics.
Pentium 4	2000	1300–3800	4 GB	Hyperthreading; more SSE instr.
Core Duo	2006	1600–3200	4 GB	Dual cores on a single die.
Core 2	2006	1200–3200	64 GB	64-bit quad core architecture.
Core i7	2011	1100–3300	24 GB	Integrated graphics processor.
Alder Lake	2021	1400–5300	128 GB	Heterogeneous cores (P and E).

Based on the book, [Tanenbaum & Austin, 2013, Fig. 1-11], and own additions.

Moore's Law Revisited



[https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/]

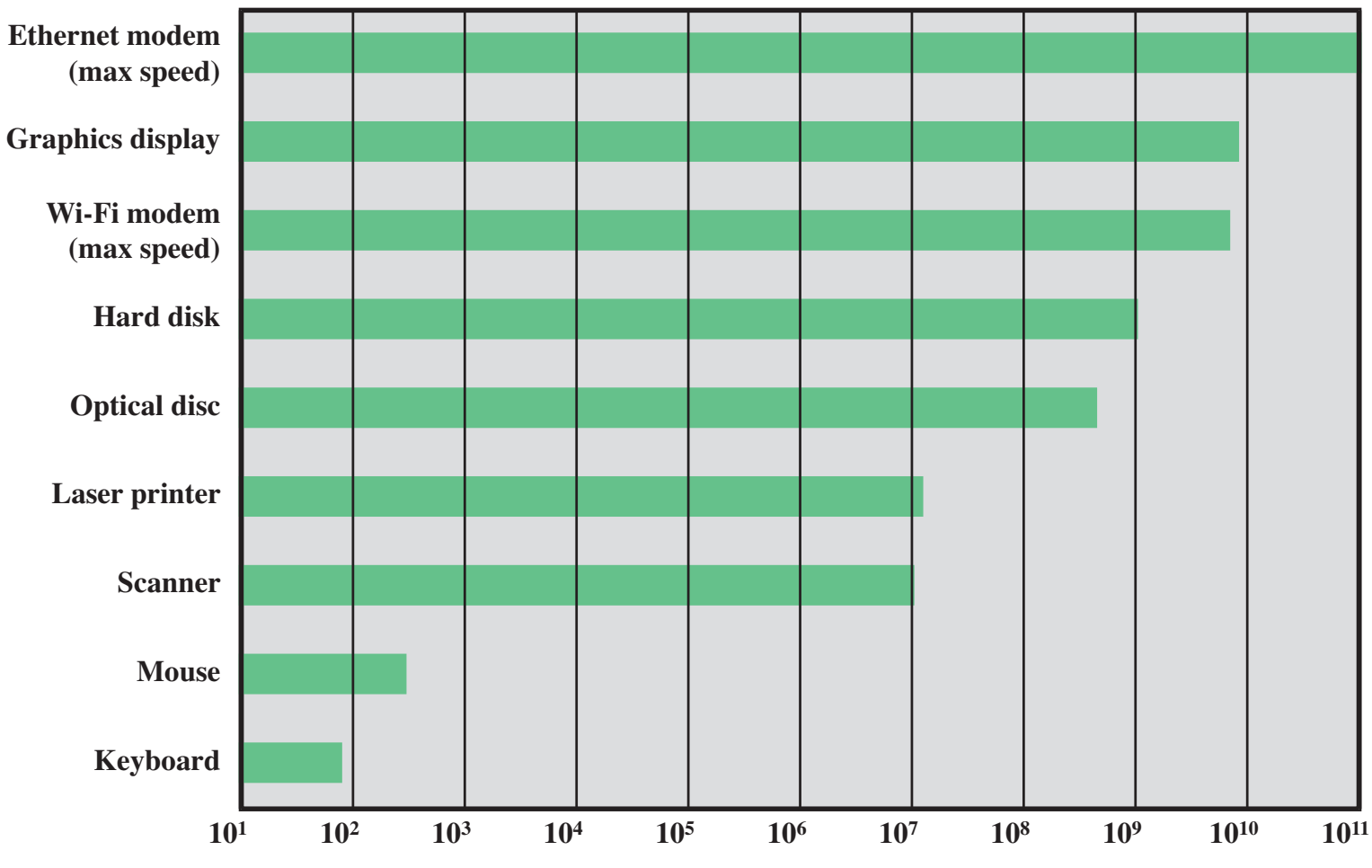
- Course Overview
- Motivation
- Gates, Memory, Chips, and a Bit of History
- Performance Concepts

How to Assess CPU Performance?

- What are we optimizing for? What is the use case?
- It is not just about clock speed:
  - Bandwidth
  - Latency
  - Instruction set
  - Power consumption / heat emission
  - Package size
  - Security features
  - Reliability
  - ...
- Example techniques to speed up a CPU:
  - Pipelining. Analogous to a factory assembly line.
  - Branch prediction and speculative execution.
  - Superscalar execution.
    E.g., multiple circuits for adding numbers.
  - Data flow analysis.
    Reorder instructions if they do not depend on each other.

Performance Balance

- CPUs have become incredibly fast.
- The memory technologies have not scaled as well.
- Different IO devices have different needs for bandwidth/latency.



Multiple Cores and Execution Speed

- Much of the current performance increase comes from adding more CPU cores.
- Utilizing many cores is non-trivial.
- Amdahl's Law: potential speed-up when using multiple cores.

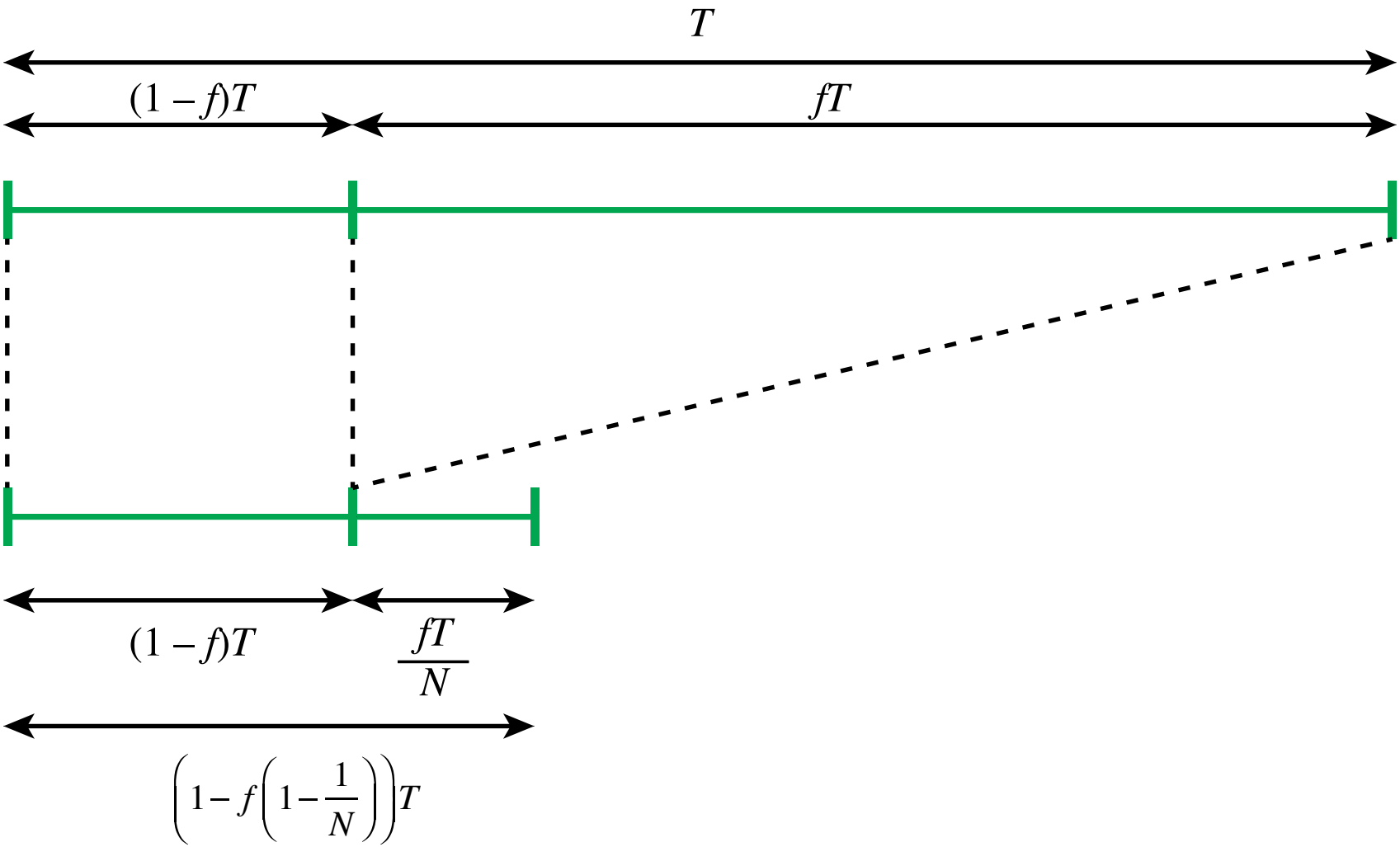
Multiple Cores and Execution Speed

- Much of the current performance increase comes from adding more CPU cores.
- Utilizing many cores is non-trivial.
- Amdahl's Law: potential speed-up when using multiple cores.
  - Let f be the fraction of a program that we can parallelize (to infinite degree).
  - Let T be the execution time on one processor.
  - Let N be the number of processors available.

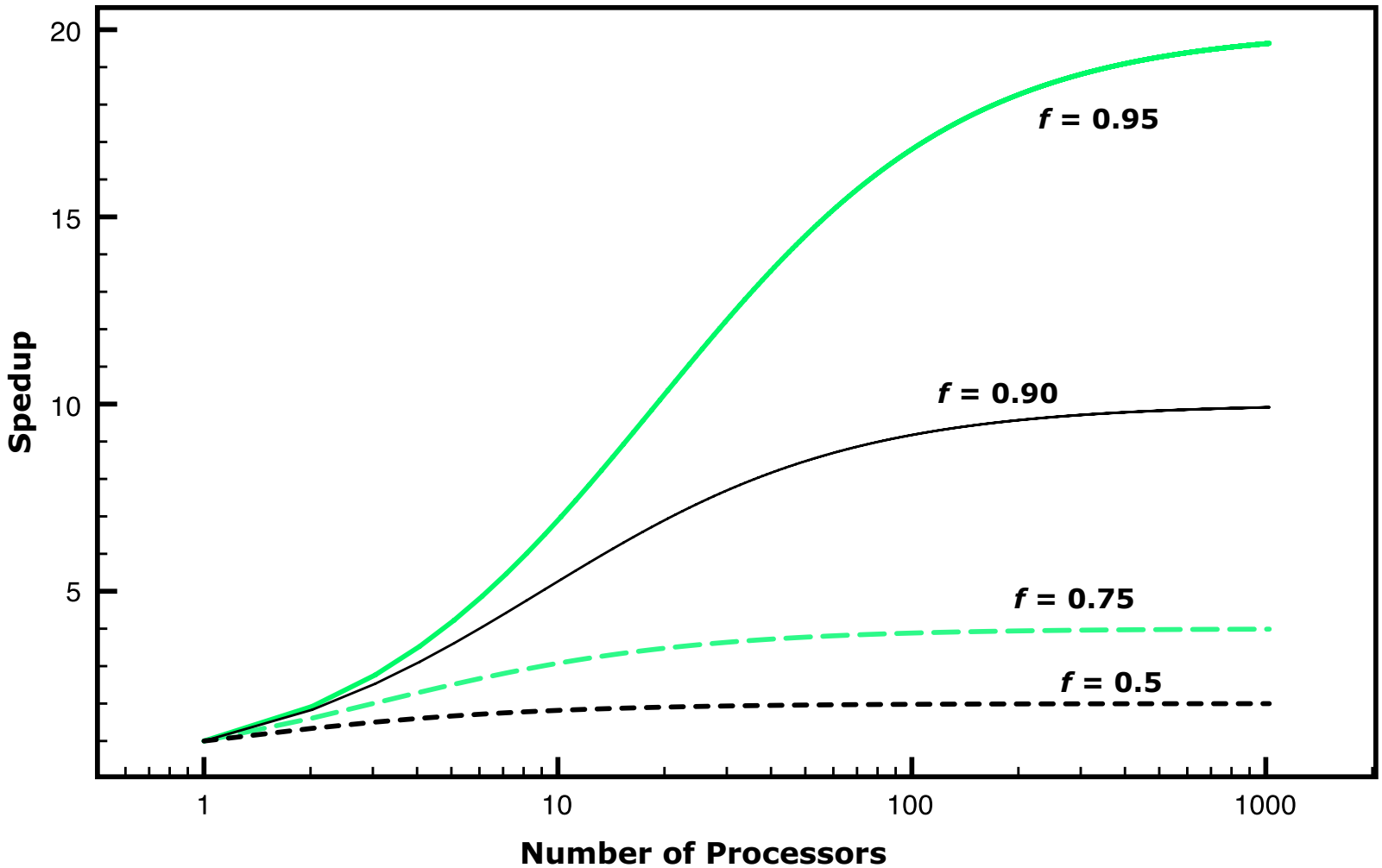
$$\begin{aligned}\text{speedup} &= \frac{\text{execution time on 1 processor}}{\text{execution time on } N \text{ processors}} \\ &= \frac{T \cdot (1-f) + T \cdot f}{T \cdot (1-f) + \frac{T \cdot f}{N}} = \frac{1}{(1-f) + \frac{f}{N}}\end{aligned}$$

Amdahl's Law

$$\text{speedup} = \frac{1}{(1-f) + \frac{f}{N}}$$



Amdahl's Law



- When f is small, there is little gain from more processors.
- When $N \to \infty$, the speedup is still bound by $\frac{1}{1-f}$.

Performance Assessment

- Basic theoretical measures:
  - Clock speed/rate
  - Millions of instructions per second (MIPS).
  - Millions of floating-point operations per second (MFLOPS). (Note: “1 FLOPS”, not “1 FLOP”)
- Benchmarks: more realistic, but depends on compiler quality.

Clock Speed

- Every step of a CPU is orchestrated by a system clock.
- Clock pulses are generated by a quartz crystal and converted into a digital signal.

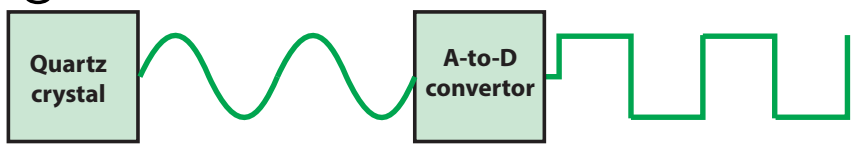


Figure 2.5 System Clock

- Typically operations start at the beginning of a pulse.
- One full pulse is a clock cycle,
  and its duration is the cycle time.
- The physical layout of a CPU limits the clock speed.
- Instructions may require different amount of clock cycles to complete.
- Different parts of the computer may have their own clocks, at different rates.

Instruction Execution Rate

- Let f be the clock frequency, so the cycle time is $\tau = \frac{1}{f}$.
- Let I_c be the total number of instructions executed in some program.
- Let I_i be the number of instructions of type i executed in that program.
- Let CPI_i be the number of cycles it takes for instructions of type i to execute.
- We can make a measure: average number of Cycles Per Instruction (CPI).

$$\text{CPI} = \frac{\sum_i \text{CPI}_i \cdot I_i}{I_c}$$

- The time to execute the program is then

$$T = I_c \cdot \text{CPI} \cdot \tau$$

(General tip for this course: be strict with units in calculations)

Instruction Execution Rate

Let's make it a bit more precise:

- Instructions may have memory accesses before/after execution.
- Memory access may be comparably slow.
- Let m be the number of memory accesses during program execution.
- Let k be the ratio between memory cycle time and CPU cycle time.
- Let p be the number cycles needed to decode and execute instructions, on average.
- Then $T = I_c \cdot (p + m \cdot k) \cdot \tau$ (but this is still rather simplified)

Instruction Execution Rate

Let's make it a bit more precise:

- Instructions may have memory accesses before/after execution.
- Memory access may be comparably slow.
- Let m be the number of memory accesses during program execution.
- Let k be the ratio between memory cycle time and CPU cycle time.
- Let p be the number cycles needed to decode and execute instructions, on average.
- Then $T = I_c \cdot (p + m \cdot k) \cdot \tau$ (but this is still rather simplified)
- Relation to system attributes:

	I_c	p	m	k	τ
Instruction set architecture	•	•			
Compiler	•	•	•		
Processor implementation		•			•
Cache and memory hierarchy				•	•

Instruction Execution Rate

More commonly we simply count the rate of instructions:

$$\text{MIPS rate} = \frac{I_c}{T \cdot 10^6} = \frac{f}{\text{CPI} \cdot 10^6}$$

$$\text{FLOPS rate} = \frac{\text{number of floating-point operations executed}}{T \cdot 10^6}$$

MIPS and ISA

Consider the statement:

```
A = B + C
```

On one machine:

```
add    mem(B), mem(C), mem(A)
```

Another machine:

```
load   mem(B), reg(1);
load   mem(C), reg(2);
add    reg(1), reg(2), reg(3);
store  reg(3), mem(A);
```

They may take the exact same time, but the second machine has a higher MIPS rate. Is it then faster?

Benchmarks

Characteristics:

- Programs written in high-level language, to make them portable between machines.
- Representative for a particular domain, e.g., system programming, numerical programming, commercial programs.
- Can be measured easily.
- Are widely distributed.

Example: SPEC Benchmarks

- Systems Performance Evaluation Corporation
- SPEC CPU2006: processor intensive applications.
- SPECjbb2000: Java Business Benchmark
- ...

"It's Difficult to Make Predictions, Especially About the Future"

- *"Computers in the future will weigh no more than 1.5 tons."*
  — Popular Mechanics, 1949
- *"I think there is a world market for maybe five computers."*
  — Thomas J. Watson, President of IBM, 1943
- *"I can assure you that data processing is a fad that won't last out the year."*
  — Editor in charge of business books, Prentice Hall, 1957
- *"Transmission of documents via telephone wires is possible in principle, but the apparatus required is so expensive that it will never become a practical proposition."*
  – Dennis Gabor, 1962
- *"There is no reason for any individual to have a computer in his home."*
  — Ken Olsen, co-founder of Digital Equipment Corporation (DEC), 1977
- "640 KiB *should be enough for anyone."*
  — Bill Gates, 1981
- *"Linux is NOT portable"*
  — Linus Torvalds, 1991.